

Slave Configuration

SSH Method: (Port 22)

Setup on Slave Machine:

Launch an EC2 instance → Name (slave-1) → SSH Login → install java on slave machine

```
# amazon-linux-extras install java-openjdk11 -y
```

Create a remote root directory for slave machine:

Eg: /mnt/jenkins-slave

Make sure you give full permission to /mnt directory

```
chmod -R 777 /mnt
```

Setup on Jenkins:

Open Jenkins dashboard: [Setup jenkins URL if not setup with correct public IP of master machine]

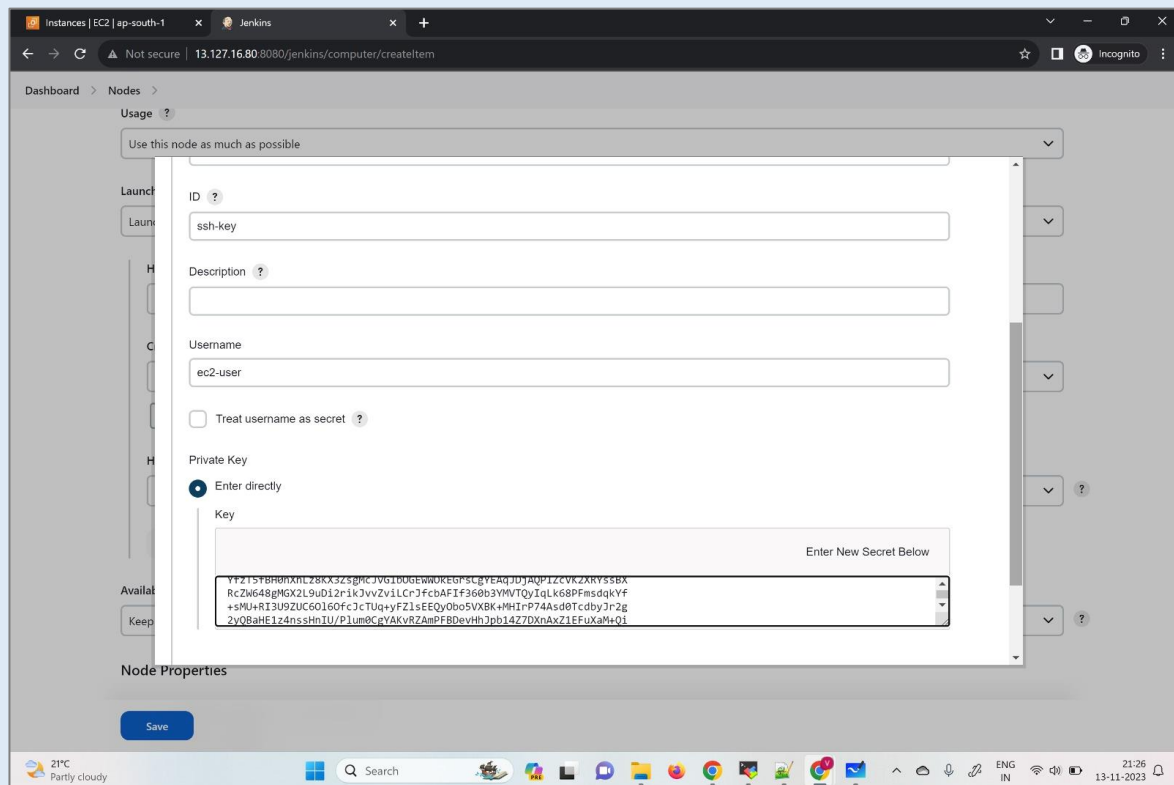
Dashboard → Manage Jenkins → Nodes → New Node → Add name (slave-1) → Select > Permanent

Agent → Create → Number of Executors (eg: 5) → Remote root directory [/mnt/Jenkins-

slave] → labels (eg: qa) → Launch method > Launch agent via SSH.. → Host (enter the private IP or DNS of slave machine) → Select Credentials →

Follow below steps to add new credentials or else skip this part if credentials are already added:

Click on Add → Kind > SSH username with Private key → ID (eg: ssh-login) → Add description → Username > ec2-user → Select Enter Directly → click on Add → Now copy and paste .pem file under Enter new secret below field → click on add



→ For Host key verification strategy, select

Host Key Verification Strategy ?

Manually trusted key Verification Strategy

☒ Require manual verification of initial connection ?

Instances | EC2 | ap-south-1 x Jenkins x +

Not secure | 13.127.16.80:8080/jenkins/computer/createItem

Dashboard > Nodes >

Host ?

172.31.46.147

Credentials ?

ec2-user (ssh-login)

Add

Host Key Verification Strategy ?

Manually trusted key Verification Strategy

☒ Require manual verification of initial connection ?

Advanced

Availability ?

Keep this agent online as much as possible

Node Properties

☐ Disable deferred wipeout on this node ?

☐ Environment variables

☐ Tool Locations

Save

21°C Partly cloudy

Search

ENG IN

21:27 13-11-2023

→ Click on **Save**

Now, Open slave-1 node and click on → Trust SSH Host Key

Status

Delete Agent

Configure

Build History

Load Statistics

Log

Trust SSH Host Key

Agent slave-1

slave-1

This node is being launched. [See log for more details](#)

Labels

qa

Projects tied to slave-1

None

→ then click on **Yes**

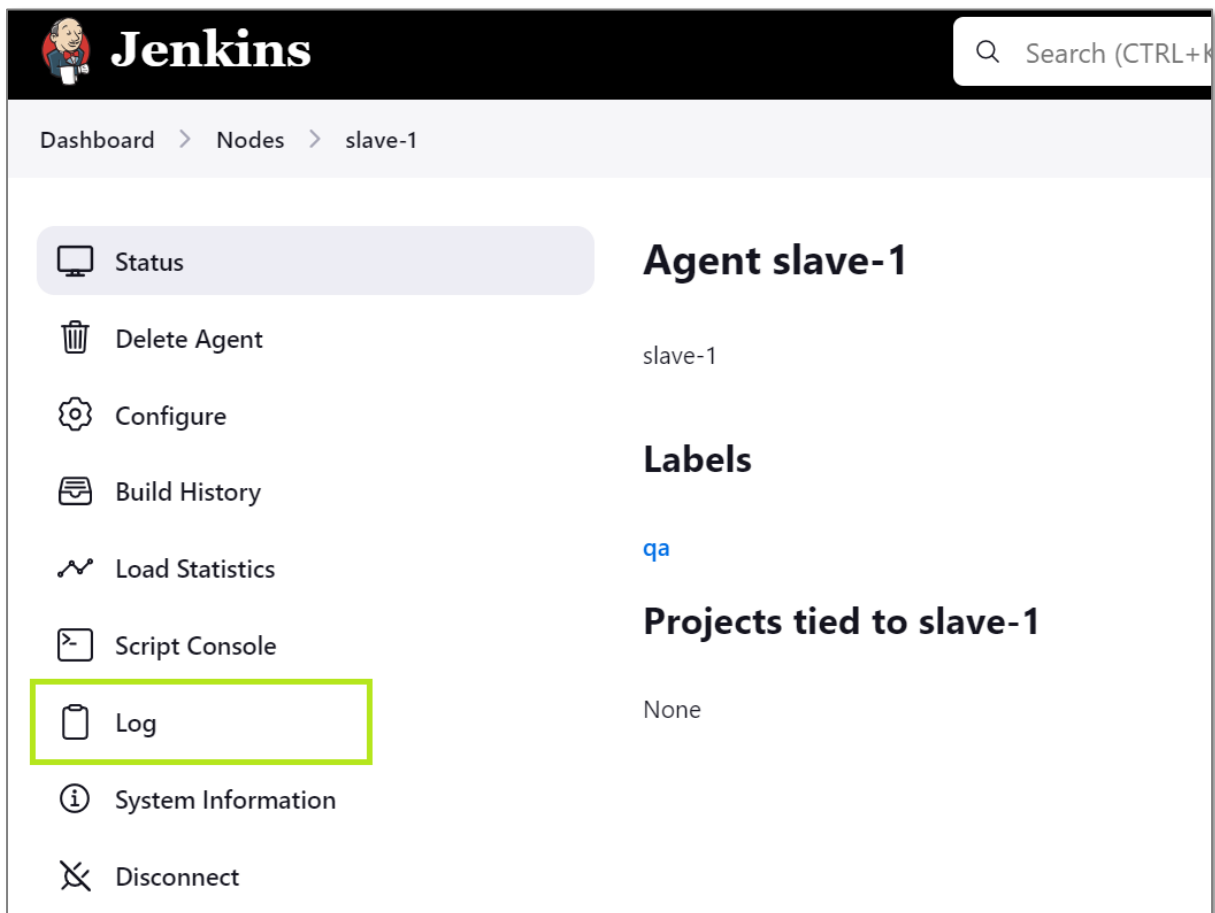


Your slave should be connected now.

Note: Try Launch/Relaunch agent option available under node if you agent is still not connected.



You can check logs here for troubleshooting:

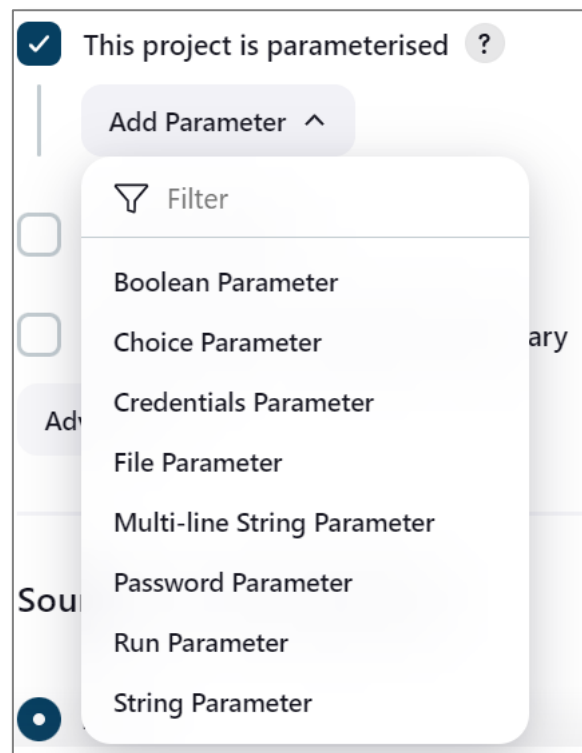


> Try to launch one more slave (slave-2) with above steps.

Parameter Passing in Jenkins Job:

How to pass parameters in ongoing job?

Dashboard → New Item → Add name (Eg: parameters) → Freestyle Project → OK → Select option > This project is parameterized → Add parameter →



Select type of parameter of your choice from the dropdown.

→ eg: Boolean Parameter → add name (eg: VALUE) → Select > Set by default

→ Add one more parameter → eg: Choice Parameter → Name (eg: ENV) → Choices >

QA
DEV
UAT
PROD

→ Add parameter > String Parameter → Name (eg: CLASS) → Default Value (eg: VELOCITY)

→ **Apply + Save** → Build with Parameter → whatever we select here will be passed to our job when we click on Build

To call the parameters in shell >>

Go to Configure → Build Step → Execute Shell →

```
echo $VALUE  
echo $ENV  
echo $CLASS
```

→ **Apply + Save**

→ Click on **Build with parameter** → add values to parameter → click on **Build**

PIPELINE JOB – (Groovy Sandbox)

- We can run temporary updates.
- For complex activities.
- We can run specific steps.
- We write pipeline/job in the form of script:
 - Scripted pipeline - Mostly Developers
 - **Declarative pipeline** - Mostly DevOps Engineers
- Two methods to write script:
 1. Create a job in Jenkins and write script inside it.
 2. Keep a script in at GitHub and run inside Jenkins.

Let's create a job first:

Dashboard → New Item → Add name (eg: 1st-pipeline) → Pipeline (~~Freestyle Project~~)

**Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

→ OK → Pipeline → Definition > Pipeline script → Write your script here under script section

Pipeline
Definition
Pipeline script
Script ?
1 write your script here.... try sample Pipeline...
☒ Use Groovy Sandbox ?

Sample Script:

```
pipeline {  
    agent any  
    stages {
```

```
stage ("1st-stage") {  
    steps {  
        echo "This is Stage-1"  
    }  
}  
  
stage ("2nd-stage") {  
    steps {  
        echo "This is Stage-2"  
    }  
}  
  
stage ("3rd-stage") {  
    steps {  
        echo "This is Stage-3"  
    }  
}  
}
```

→ **Apply + Save** → **Build now**

To run a Specific stage in job:

Open successful build (In **Build History**) → Click on Restart from stage and select stage

 Status

 Changes

 Console Output

 Edit Build Information

 Delete build '#2'

 Restart from Stage

Restart #2 from Stage

Stage Name

1st-stage

1st-stage

2nd-stage

3rd-stage

Run

To run temporary changes in the Job:

Open successful build (In **Build History**) → and then click on Replay, make changes to script and click on Run (Note: It will not affect original script)

Status

Changes

Console Output

Edit Build Information

Delete build '#2'

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Replay #2

Allows you to replay a Pipeline build with a modified script. If any load steps were run, you can also modify the scripts they loaded.

Main Script

```
1 pipeline {
2
3     agent any
4
5     stages {
6
7         stage ("1st-stage") {
8
9             steps {
10                 echo "This is Stage-1"
11             }
12         }
13
14     }
15
16     stage ("2nd-stage") {
17
18         steps {
```

Run

Commenting in groovy Sandbox:

Single line comment

//

```
1 pipeline {
2
3     agent any
4
5     stages {
6
7         // stage ("1st-stage") {
8
9             steps {
10                 echo "This is Stage-1"
11             }
12         }
13     }
```

Multiline comment

/*
..... */

```
6
7     /* stage ("1st-stage") {
8
9         steps {
10             echo "This is Stage-1"
11         }
12     } */
13
14     stage ("2nd-stage") {
15
16         steps {
```